

Experiment No. 8

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Theory:

Ganache is a personal Ethereum blockchain used for testing and development. It allows developers to deploy contracts, develop decentralized applications (DApps), and run tests in a safe and deterministic environment.

Ganache provides:

- Instant mining of blocks
- Pre-funded test accounts
- Transaction history view
- Graphical interface to monitor blockchain activities

It is part of the Truffle Suite and is commonly used with Remix and Metamask for smart contract deployment.

Connection between Ganache, MetaMask, and Remix IDE

These three tools work together to create a complete blockchain workflow:

1. Ganache → MetaMask

- Ganache provides **local blockchain + accounts**
- MetaMask connects using:
 - RPC URL (e.g., 127.0.0.1:7545)
- Ganache accounts are imported into MetaMask using private keys
- MetaMask can now use Ganache accounts

2. MetaMask → Remix IDE

- In Remix, select:
👉 **Injected Provider - MetaMask**
- Remix connects to MetaMask wallet
- Remix uses MetaMask account to perform transactions

3. Remix IDE → Ganache

- When you deploy a contract:
 - Remix → sends request to MetaMask
 - MetaMask → signs transaction
 - Ganache → executes transaction

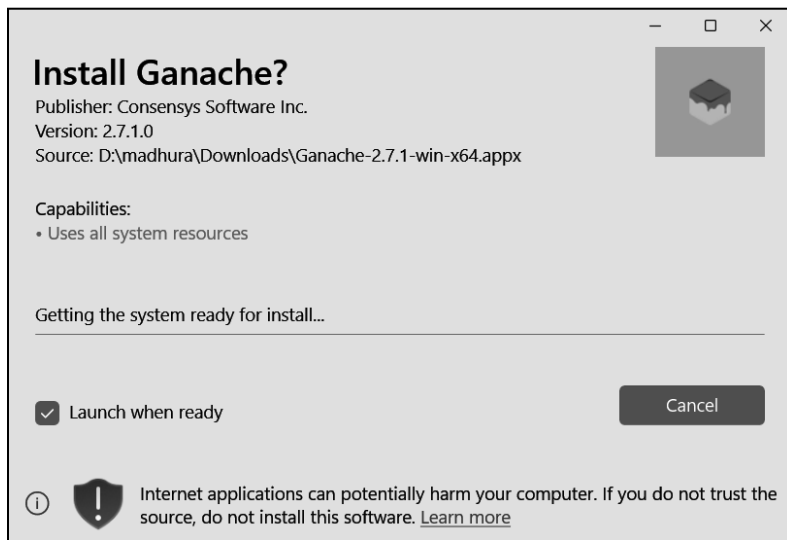
The contract gets deployed on the local blockchain.

Advantages of this setup

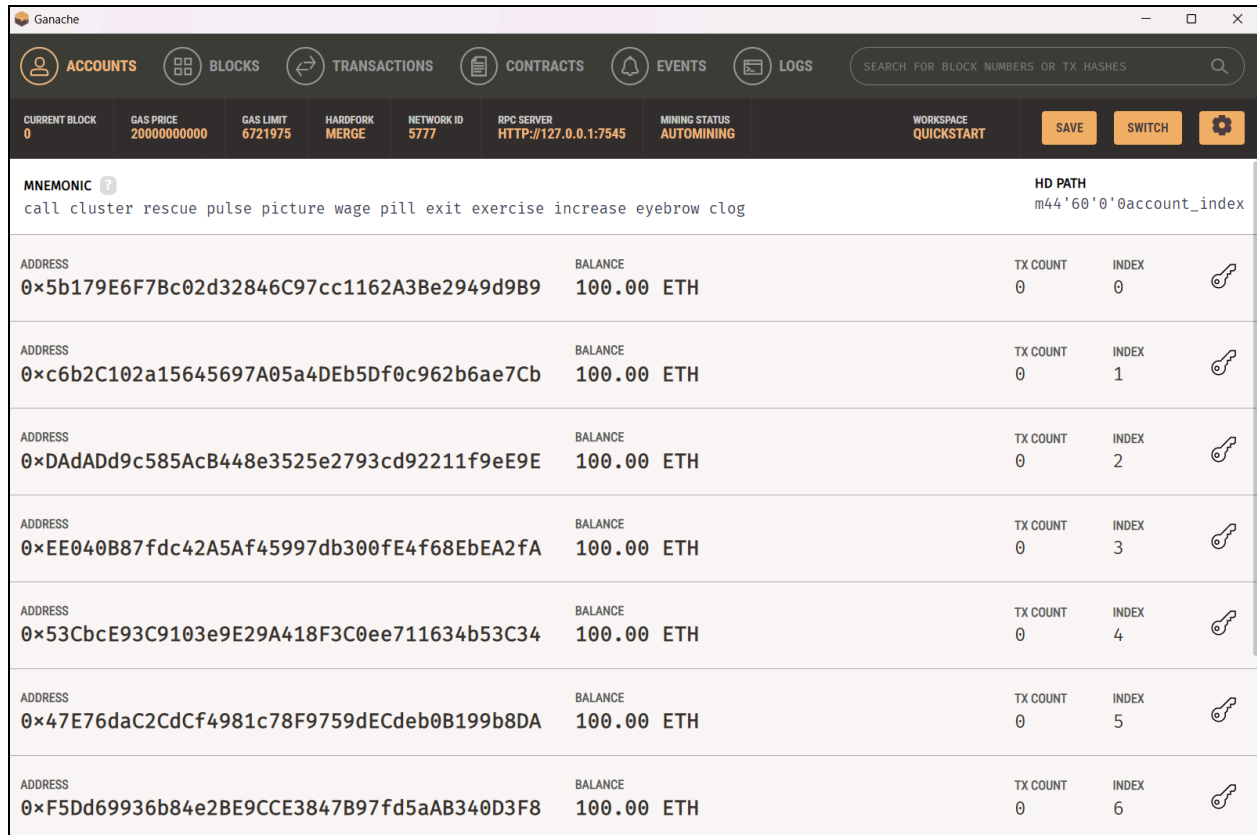
- No real money required
- Fast execution
- Easy debugging
- Safe testing environment

Implementation:

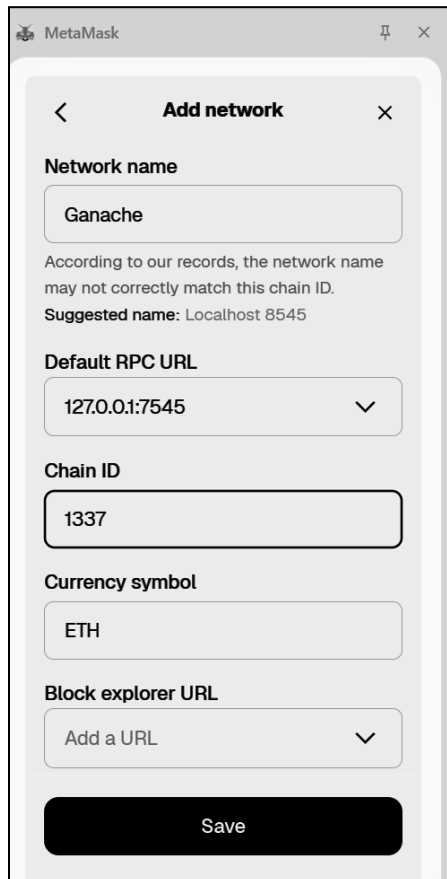
Installing Ganache in system-



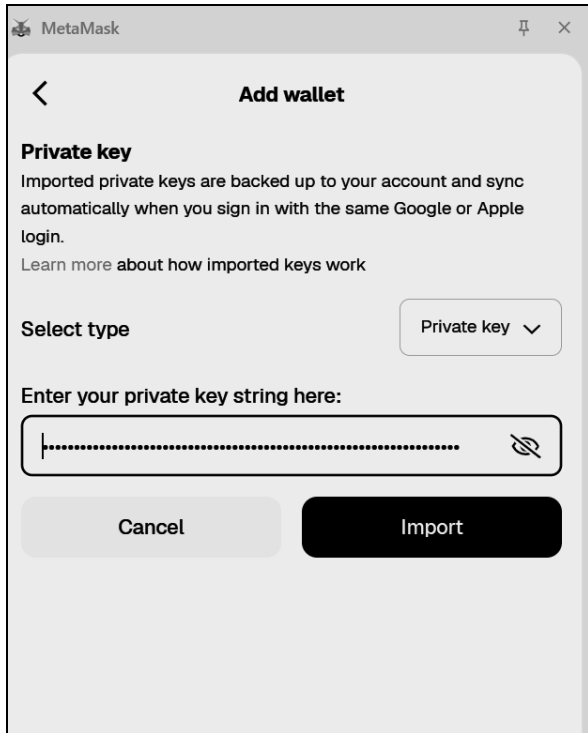
After installing Ganache, clicking on quickstart we get 10 accounts with 100ETH each-



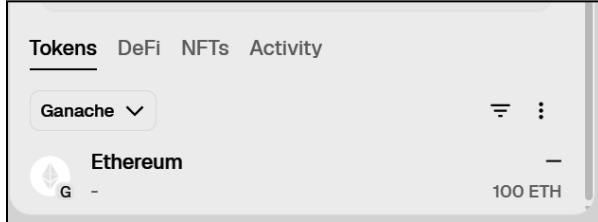
In Matamask Adding Ganache network-



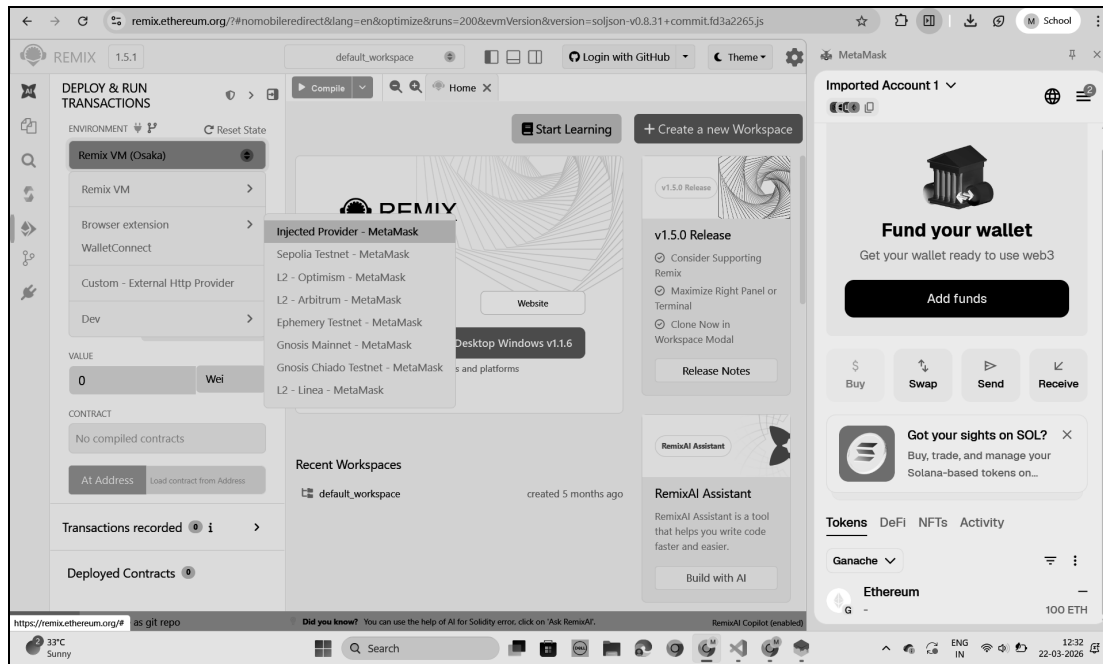
Copying the private key of account 0 and importing that account in metamask-



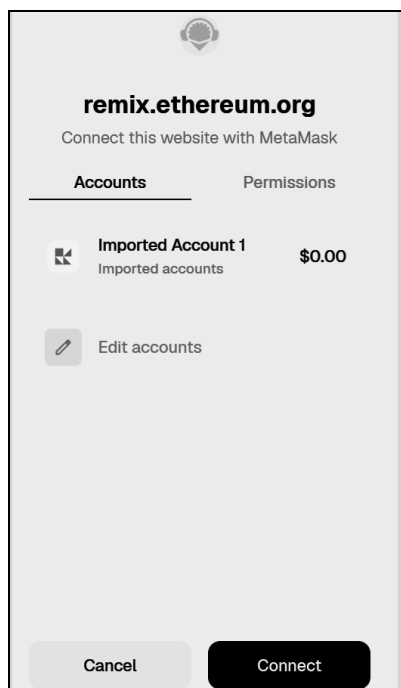
Account imported successfully balance is 100 ETH-



Selecting injected provider - metamask environment in remix ide-



Connection with remix ide-



Deploying contracts-

Contract 1- TrustScore.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.17;
```

```
contract TrustScore {
    // Store trust scores for each user
    mapping(address => uint256) public trustScores;
    address public admin;
```

```
    address public loanManagerAddress;
    uint256 public initialScore = 50; // Initial score for
    new users
```

```
    constructor() {
        admin = msg.sender;
    }
```

```

// Set LoanManager address (admin only)
function setLoanManagerAddress(address
_loanManagerAddress) public {
    require(msg.sender == admin, "Only admin");
    loanManagerAddress = _loanManagerAddress;
}

// Admin or LoanManager can update scores
function updateScore(address user, uint256 score)
public {
    require(msg.sender == admin || msg.sender ==
loanManagerAddress, "Only admin or
LoanManager");
    require(score <= 100, "Score must be 0-100");
    trustScores[user] = score;
}

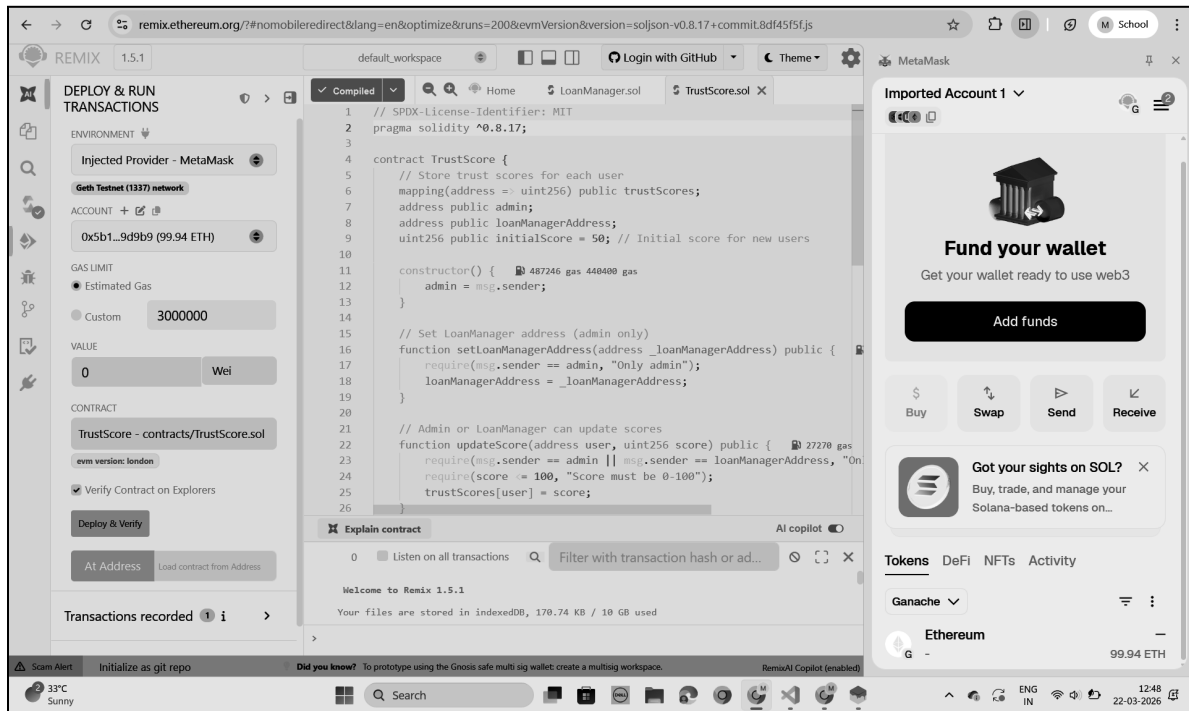
```

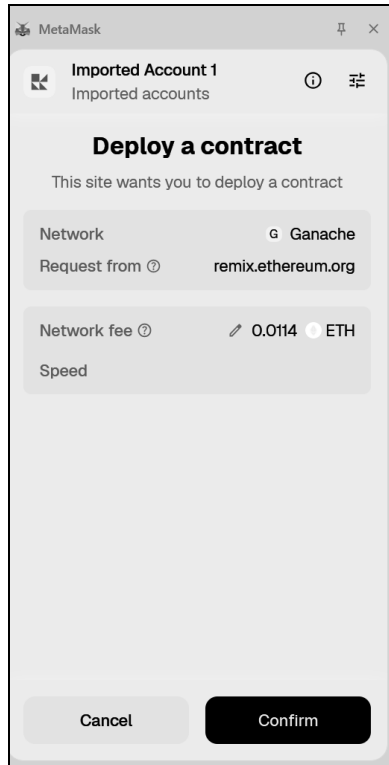
```

// Set initial score for new users (admin only)
function setInitialScore(uint256 _initialScore)
public {
    require(msg.sender == admin, "Only admin");
    require(_initialScore <= 100, "Score must be
0-100");
    initialScore = _initialScore;
}

// Anyone can check a score - returns initialScore if
not set
function getScore(address user) public view
returns (uint256) {
    uint256 score = trustScores[user];
    // If score is 0, return initial score (user hasn't
been explicitly set)
    return score > 0 ? score : initialScore;
}
}

```





Contract 2- LoanManager.sol

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.17;

import "./TrustScore.sol";

```
interface ITrustScore {
    function getScore(address user) external view
    returns (uint256);
    function updateScore(address user, uint256 score)
    external;
}
```

```
contract LoanManager {
```

```
    TrustScore public trustScoreContract;
```

```
    address public admin;
```

```
    uint256 public minTrustScore;
```

```
    // Trust score rewards/penalties
```

```
    uint256 public repaymentBonus = 5;    // +5
    points for successful repayment
```

```
    uint256 public defaultPenalty = 10;    // -10 points
    for defaulted loan
```

```
    // Safety pool address for collecting fees
```

```
    address public safetyPoolAddress;
```

```
    constructor(address _trustScoreAddress) {
```

```
        trustScoreContract =
```

```
        TrustScore(_trustScoreAddress);
```

```
        admin = msg.sender;
```

```
        minTrustScore = 40; // Default minimum trust
```

```
score
```

```
    }
```

```
    // Set safety pool address (admin only)
```

```
    function setSafetyPoolAddress(address
```

```
    _safetyPoolAddress) public {
```

```
        require(msg.sender == admin, "Only admin");
```

```
        safetyPoolAddress = _safetyPoolAddress;
```

```
    }
```

```
    struct Loan {
```

```
        address borrower;
```

```
        address lender;
```

```
        uint256 amount;
```

```
        uint256 interestRate; // Interest rate in basis
        points (e.g., 500 = 5%)
```

```
        uint256 safetyFee;    // Safety fee in basis points
```

```
        uint256 duration;
```

```
        uint256 createdAt;
```

```
        bool funded;
```

```
        bool repaid;
```

```
        bool withdrawn;
```

```
        bool defaulted;
```

```

    }

    Loan[] public loans;

    event LoanRequested(uint256 loanId, address
    borrower, uint256 amount, uint256 interestRate);
    event LoanFunded(uint256 loanId, address lender);
    event LoanWithdrawn(uint256 loanId);
    event LoanRepaid(uint256 loanId, uint256
    totalRepaid, uint256 interest);
    event LoanDefaulted(uint256 loanId, address
    borrower);
    event TrustScoreIncreased(address borrower,
    uint256 amount);
    event TrustScoreDecreased(address borrower,
    uint256 amount);
    event MinTrustScoreUpdated(uint256 newScore);

    // Calculate interest rate based on trust score
    // Higher trust score = lower interest rate
    function getInterestRate(uint256 trustScore) public
    pure returns (uint256) {
        if (trustScore >= 90) return 300;    // 3% for
    excellent credit
        if (trustScore >= 75) return 500;    // 5% for
    good credit
        if (trustScore >= 60) return 800;    // 8% for fair
    credit
        if (trustScore >= 50) return 1200;    // 12% for
    poor credit
        return 1500;                        // 15% for very
    poor credit
    }

    // Get maximum borrow limit based on trust score
    (in wei)
    // Higher trust = larger loans allowed
    function getMaxBorrowLimit(uint256 trustScore)
    public pure returns (uint256) {
        if (trustScore >= 90) return 10 ether;    //
    Excellent: up to 10 ETH
        if (trustScore >= 75) return 5 ether;    // Good:
    up to 5 ETH
        if (trustScore >= 60) return 2 ether;    // Fair: up
    to 2 ETH
        if (trustScore >= 50) return 0.5 ether;    // Poor:
    up to 0.5 ETH
        return 0.1 ether;                    // Very poor: up
    to 0.1 ETH
    }

    // Get safety fee based on trust score (in basis
    points)

```

```

    // Lower trust = higher safety fee to protect lenders
    function getSafetyFee(uint256 trustScore) public
    pure returns (uint256) {
        if (trustScore >= 90) return 50;    // 0.5% for
    excellent credit
        if (trustScore >= 75) return 100;    // 1% for
    good credit
        if (trustScore >= 60) return 200;    // 2% for fair
    credit
        if (trustScore >= 50) return 400;    // 4% for
    poor credit
        return 600;                        // 6% for very poor
    credit
    }

    // Get borrower's current limit info
    function getBorrowerLimits(address borrower)
    public view returns (
        uint256 trustScore,
        uint256 maxBorrowLimit,
        uint256 interestRate,
        uint256 safetyFee
    ) {
        trustScore =
    trustScoreContract.getScore(borrower);
        maxBorrowLimit =
    getMaxBorrowLimit(trustScore);
        interestRate = getInterestRate(trustScore);
        safetyFee = getSafetyFee(trustScore);
    }

    // Calculate total repayment amount (principal +
    interest + safety fee)
    function calculateRepayment(uint256 loanId)
    public view returns (uint256) {
        Loan memory loan = loans[loanId];
        uint256 interest = (loan.amount *
    loan.interestRate) / 10000;
        uint256 safetyFeeAmount = (loan.amount *
    loan.safetyFee) / 10000;
        return loan.amount + interest +
    safetyFeeAmount;
    }

    // Get breakdown of repayment components
    function getRepaymentBreakdown(uint256 loanId)
    public view returns (
        uint256 principal,
        uint256 interest,
        uint256 safetyFeeAmount,
        uint256 total
    ) {
        Loan memory loan = loans[loanId];

```

```

        principal = loan.amount;
        interest = (loan.amount * loan.interestRate) /
10000;
        safetyFeeAmount = (loan.amount *
loan.safetyFee) / 10000;
        total = principal + interest + safetyFeeAmount;
    }

    // Admin function to update minimum trust score
    function setMinTrustScore(uint256 _minScore)
public {
    require(msg.sender == admin, "Only admin can
set minimum trust score");
    require(_minScore <= 100, "Score must be
0-100");
    minTrustScore = _minScore;
    emit MinTrustScoreUpdated(_minScore);
}

    // Set trust score reward for repayment
    function setRepaymentBonus(uint256 _bonus)
public {
    require(msg.sender == admin, "Only admin");
    repaymentBonus = _bonus;
}

    // Set trust score penalty for default
    function setDefaultPenalty(uint256 _penalty)
public {
    require(msg.sender == admin, "Only admin");
    defaultPenalty = _penalty;
}

    // Check if a loan is overdue
    function isLoanOverdue(uint256 loanId) public
view returns (bool) {
        Loan memory loan = loans[loanId];
        if (!loan.withdrawn || loan.repaid ||
loan.defaulted) {
            return false;
        }
        return block.timestamp > loan.createdAt +
loan.duration;
    }

    // Mark loan as defaulted (called by admin or
lender)
    function markLoanDefaulted(uint256 loanId)
public {
        Loan storage loan = loans[loanId];
        require(!loan.defaulted && !loan.repaid, "Loan
already finalized");

```

```

        require(isLoanOverdue(loanId), "Loan is not
overdue");
        require(msg.sender == loan.lender || msg.sender
== admin, "Only lender or admin can mark default");

        loan.defaulted = true;

        // Decrease borrower's trust score
        uint256 currentScore =
trustScoreContract.getScore(loan.borrower);
        uint256 newScore = currentScore >
defaultPenalty ? currentScore - defaultPenalty : 0;
        trustScoreContract.updateScore(loan.borrower,
newScore);

        emit LoanDefaulted(loanId, loan.borrower);
        emit TrustScoreDecreased(loan.borrower,
defaultPenalty);
    }

    // -----
    // BORROWER CREATES LOAN REQUEST
    // -----
    function createLoan(uint256 amount, uint256
duration) public {

        uint256 score =
trustScoreContract.getScore(msg.sender);
        require(score >= minTrustScore, "Trust score
too low");

        // Enforce maximum borrow limit based on trust
score
        uint256 maxLimit =
getMaxBorrowLimit(score);
        require(amount <= maxLimit, "Amount exceeds
your borrow limit");

        uint256 interestRate = getInterestRate(score);
        uint256 safetyFee = getSafetyFee(score);

        loans.push(
            Loan({
                borrower: msg.sender,
                lender: address(0),
                amount: amount,
                interestRate: interestRate,
                safetyFee: safetyFee,
                duration: duration,
                createdAt: block.timestamp,
                funded: false,
                repaid: false,
                withdrawn: false,

```

```

        defaulted: false
    })
);

    emit LoanRequested(loans.length - 1,
msg.sender, amount, interestRate);
}

// -----
// LENDER FUNDS LOAN
// -----
function fundLoan(uint256 loanId) public payable
{

    Loan storage loan = loans[loanId];

    require(!loan.funded, "Already funded");
    require(msg.value == loan.amount, "Incorrect
ETH amount");

    loan.lender = msg.sender;
    loan.funded = true;

    emit LoanFunded(loanId, msg.sender);
}

// -----
// BORROWER WITHDRAWS FUNDS
// -----
function withdrawLoan(uint256 loanId) public {

    Loan storage loan = loans[loanId];

    require(msg.sender == loan.borrower, "Not
borrower");
    require(loan.funded, "Loan not funded");
    require(!loan.withdrawn, "Already withdrawn");

    loan.withdrawn = true;

    payable(msg.sender).transfer(loan.amount);

    emit LoanWithdrawn(loanId);
}

// -----
// BORROWER REPAYS LOAN
// -----
function repayLoan(uint256 loanId) public payable
{

    Loan storage loan = loans[loanId];

```

```

    require(msg.sender == loan.borrower, "Not
borrower");
    require(loan.withdrawn, "Loan not withdrawn");
    require(!loan.repaid, "Already repaid");
    require(!loan.defaulted, "Loan is defaulted");
    require(loan.lender != address(0), "Invalid
lender address");

    // Get repayment breakdown
    (uint256 principal, uint256 interest, uint256
safetyFeeAmount, uint256 totalRepayment) =
getRepaymentBreakdown(loanId);
    require(msg.value >= totalRepayment,
"Insufficient repayment amount");

    // Transfer principal + interest to lender
    uint256 lenderAmount = principal + interest;
    payable(loan.lender).transfer(lenderAmount);

    // Transfer safety fee to safety pool (if set)
    if (safetyPoolAddress != address(0) &&
safetyFeeAmount > 0) {

        payable(safetyPoolAddress).transfer(safetyFeeAmou
nt);
    }

    loan.repaid = true;

    // Increase borrower's trust score for successful
repayment
    uint256 currentScore =
trustScoreContract.getScore(loan.borrower);
    uint256 newScore = currentScore +
repaymentBonus;
    // Cap at 100
    if (newScore > 100) newScore = 100;
    trustScoreContract.updateScore(loan.borrower,
newScore);

    emit LoanRepaid(loanId, totalRepayment,
interest);
    emit TrustScoreIncreased(loan.borrower,
repaymentBonus);

    // Return excess payment if any
    if (msg.value > totalRepayment) {
        payable(msg.sender).transfer(msg.value -
totalRepayment);
    }
}

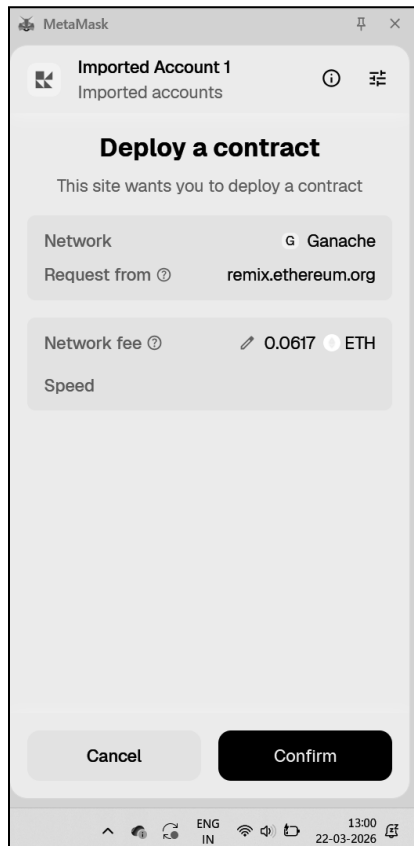
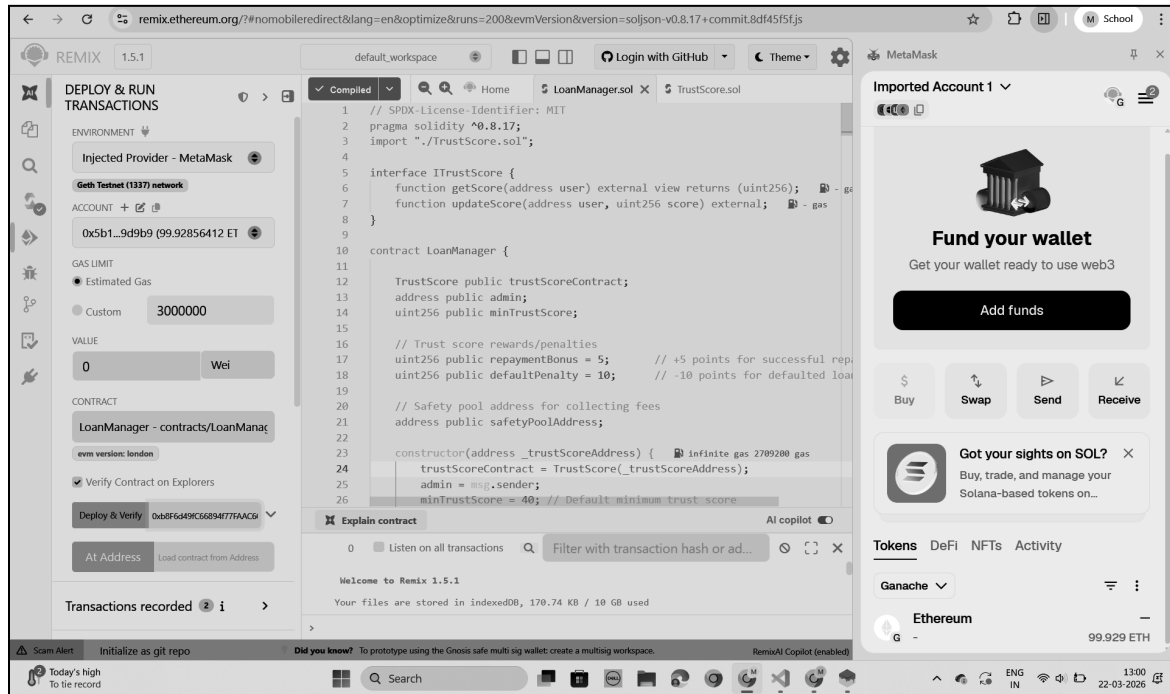
// -----

```

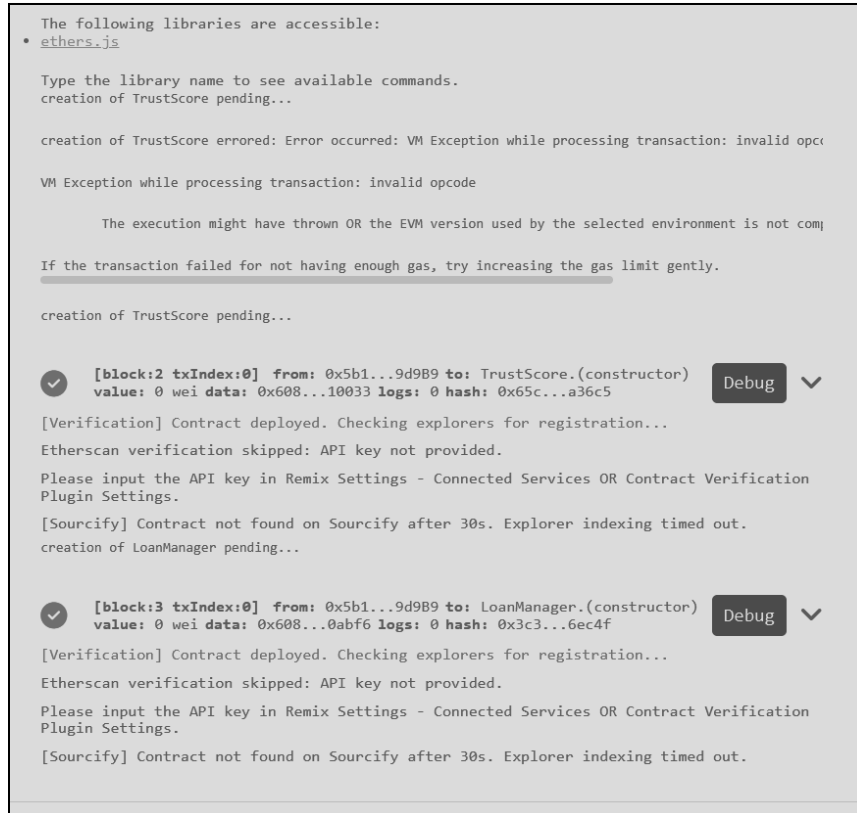
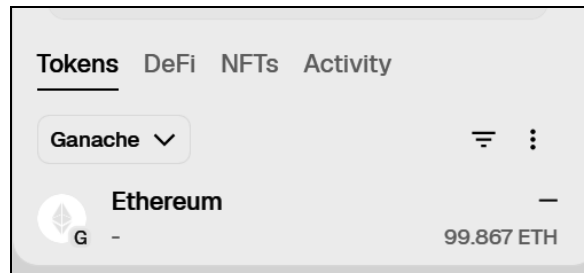
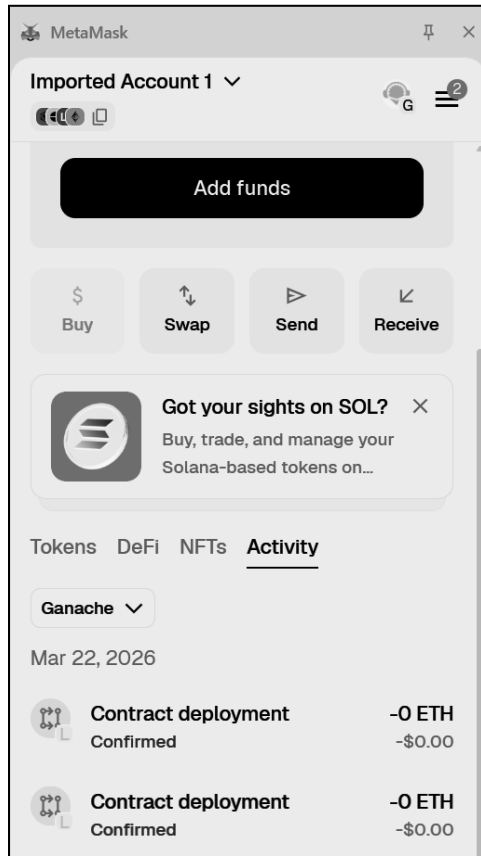
```

// GET ALL LOANS
// -----
function getLoans() public view returns (Loan[]
memory) {
                                }
                                }
                                return loans;
}
}

```



Contracts Deployed successfully-



Linking both the contracts by function- setLoanManagerAddress(LoanManager_address)

The screenshot shows the Remix IDE interface. On the left, the 'Deployed Contracts' list shows two contracts: 'TrustScore' and 'LoanManager'. The 'TrustScore' contract has a balance of 0 ETH. The 'LoanManager' contract has a balance of 0 ETH. The main panel displays the transaction logs for the deployment of TrustScore, showing the 'setLoanManagerAddress' function being called. The right sidebar shows the 'Transaction request' dialog with the network set to Ganache and the request from 'remix.ethereum.org'.

getScore(your_address) - is 50 which is default

The screenshot shows a button labeled 'initialScore' and the value '0: uint256: 50'.

Function createLoan - Loan of 2 Eth created

The screenshot shows the Remix IDE interface. On the left, the 'Deployed Contracts' list shows two contracts: 'TrustScore' and 'LoanManager'. The 'TrustScore' contract has a balance of 0 ETH. The 'LoanManager' contract has a balance of 0 ETH. The main panel displays the transaction logs for the deployment of TrustScore, showing the 'createLoan' function being called. The right sidebar shows the 'Transaction request' dialog with the network set to Ganache and the request from 'remix.ethereum.org'.

Balance became 99.86 ETH after all the transactions-

Ganache

ACCOUNTS

BLOCKS

TRANSACTIONS

CONTRACTS

EVENTS

LOGS

SEARCH FOR BLOCK NUMBERS OR TX HASHES

CURRENT BLOCK
6

GAS PRICE
2000000000

GAS LIMIT
6721975

HARDFORK
MERGE

NETWORK ID
5777

RPC SERVER
HTTP://127.0.0.1:7545

MINING STATUS
AUTOMINING

WORKSPACE
QUICKSTART

SAVE

SWITCH

MNEMONIC

call cluster rescue pulse picture wage pill exit exercise increase eyebrow clog

HD PATH

m44'60'0'0'account_index

ADDRESS

0x5b179E6F7Bc02d32846C97cc1162A3Be2949d9B9

BALANCE

99.86 ETH

TX COUNT

6

INDEX

0

ADDRESS

0xc6b2C102a15645697A05a4DEb5Df0c962b6ae7Cb

BALANCE

100.00 ETH

TX COUNT

0

INDEX

1

ADDRESS

0xDAAdD9c585AcB448e3525e2793cd92211f9eE9E

BALANCE

100.00 ETH

TX COUNT

0

INDEX

2

ADDRESS

0xEE040B87fdc42A5Af45997db300fE4f68EbEA2fA

BALANCE

100.00 ETH

TX COUNT

0

INDEX

3

ADDRESS

0x53CbcE93C9103e9E29A418F3C0ee711634b53C34

BALANCE

100.00 ETH

TX COUNT

0

INDEX

4

ADDRESS

0x47E76daC2CdCf4981c78F9759dECdeb0B199b8DA

BALANCE

100.00 ETH

TX COUNT

0

INDEX

5

ADDRESS

0xF5Dd69936b84e2BE9CCE3847B97fd5aAB340D3F8

BALANCE

100.00 ETH

TX COUNT

0

INDEX

6

ADDRESS

0x42CBc877791d28138204C48494b9CDc833f1DB56

BALANCE

100.00 ETH

TX COUNT

0

INDEX

7

Transactions in Ganache-

Ganache

ACCOUNTS

BLOCKS

TRANSACTIONS

CONTRACTS

EVENTS

LOGS

SEARCH FOR BLOCK NUMBERS OR TX HASHES

CURRENT BLOCK
6

GAS PRICE
2000000000

GAS LIMIT
6721975

HARDFORK
MERGE

NETWORK ID
5777

RPC SERVER
HTTP://127.0.0.1:7545

MINING STATUS
AUTOMINING

WORKSPACE
QUICKSTART

SAVE

SWITCH

TX HASH

0xc590512e98401b6ed38a7e789653476039a3505e91ede413d795392091c2bf03

CONTRACT CALL

FROM ADDRESS

0x5b179E6F7Bc02d32846C97cc1162A3Be2949d9B9

TO CONTRACT ADDRESS

0xa07499e663d9Acf02A8220a26C5F55eE11FFf436

GAS USED

198478

VALUE

0

TX HASH

0xaa9f9f182d9ffeee1639d11b7e6b2d7fd541ef40bd7ba0f891609daf79d11a0c0

CONTRACT CALL

FROM ADDRESS

0x5b179E6F7Bc02d32846C97cc1162A3Be2949d9B9

TO CONTRACT ADDRESS

0xa07499e663d9Acf02A8220a26C5F55eE11FFf436

GAS USED

34995

VALUE

0

TX HASH

0x484fe9c01e12f6a7780571ca44915add62f5930676d3283d6cc2ef630e20f529

CONTRACT CALL

FROM ADDRESS

0x5b179E6F7Bc02d32846C97cc1162A3Be2949d9B9

TO CONTRACT ADDRESS

0xb8F6d49fC66894f77FAAC60b4A8Ba8680770ABf6

GAS USED

46268

VALUE

0

TX HASH

0x4c1cf3ecbccf018807619e2647fac2116cf5d39bfe8532d5ab9b993faede0692

CONTRACT CREATION

FROM ADDRESS

0x5b179E6F7Bc02d32846C97cc1162A3Be2949d9B9

CREATED CONTRACT ADDRESS

0xa07499e663d9Acf02A8220a26C5F55eE11FFf436

GAS USED

3082525

VALUE

0

TX HASH

0x67e386b411dd100a3299018e28593892fb51be5a1ecca8c5beea46680e7c71b3

CONTRACT CREATION

FROM ADDRESS

0x5b179E6F7Bc02d32846C97cc1162A3Be2949d9B9

CREATED CONTRACT ADDRESS

0xb8F6d49fC66894f77FAAC60b4A8Ba8680770ABf6

GAS USED

571794

VALUE

0

No. of Blocks Mined-

ACCOUNTS

BLOCKS

TRANSACTIONS

CONTRACTS

EVENTS

LOGS

SEARCH FOR BLOCK NUMBERS OR TX HASHES

CURRENT BLOCK

6

GAS PRICE

20000000000

GAS LIMIT

6721975

HARDFORK

MERGE

NETWORK ID

5777

RPC SERVER

HTTP://127.0.0.1:7545

MINING STATUS

AUTOMINING

WORKSPACE

QUICKSTART

SAVE

SWITCH

BLOCK

6

MINED ON

2026-03-22 13:08:06

GAS USED

198478

1 TRANSACTION

BLOCK

5

MINED ON

2026-03-22 13:06:15

GAS USED

34995

1 TRANSACTION

BLOCK

4

MINED ON

2026-03-22 13:04:00

GAS USED

46268

1 TRANSACTION

BLOCK

3

MINED ON

2026-03-22 13:00:50

GAS USED

3082525

1 TRANSACTION

BLOCK

2

MINED ON

2026-03-22 12:49:22

GAS USED

571794

1 TRANSACTION

BLOCK

1

MINED ON

2026-03-22 12:44:36

GAS USED

3000000

1 TRANSACTION

BLOCK

0

MINED ON

2026-03-22 11:52:02

GAS USED

0

NO TRANSACTIONS

Conclusion:

Ganache provides a simple and powerful environment to simulate a local Ethereum blockchain. By integrating it with **Metamask** and **Remix**, developers can deploy, test, and debug smart contracts efficiently before moving to a public network.